Introduction to RAG and It's Application

Prof. Osama Abdel Raouf

Agenda

- Introduction
  - What is LLM?
  - What is RAG?
- LLM And It's Limitation
  - Why RAG is important?
- RAG Architecture
- How Does RAG Work?
- RAG vs Fine-Tuning
- Benefits Of RAG
- Applications
- Demo

Revision

What is LLM?

- A large language model (LLM) is a type of artificial intelligence program that can recognize and generate text, among other tasks.
- LLM are very large models that are **pre-trained on vast amounts** of data.
- Built on **transformer architecture** is a set of neural network that consist of an encoder and a decoder with self-attention capabilities.
- It can **perform completely different tasks** such as answering questions, summarizing documents, translating languages and completing sentences.
- Open AI's GPT-3 model has 175 billion parameters. Also it can take inputs up to 100K tokens limit.
- Moving to GPT-4 can handle up to **32,768 tokens** for

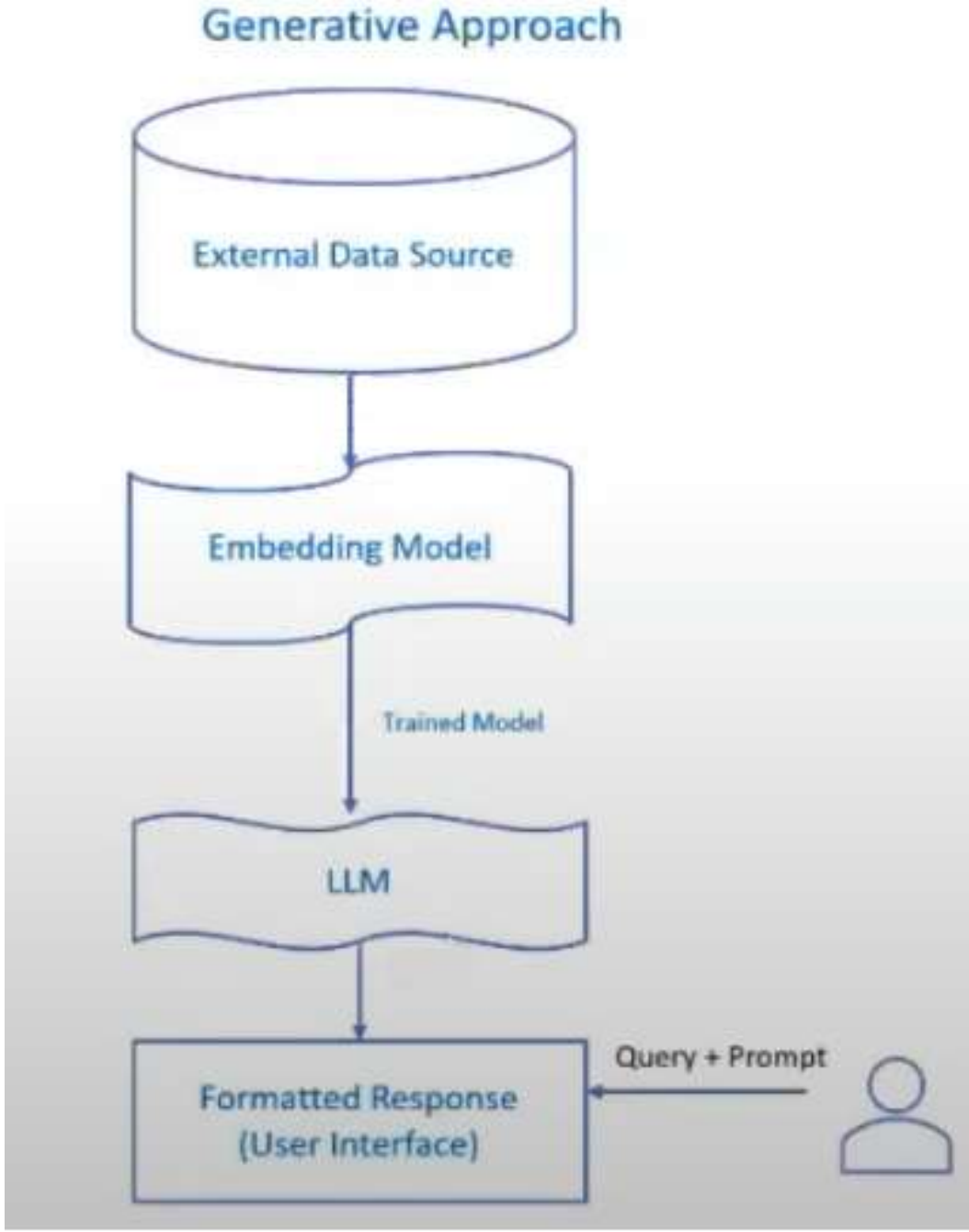
Revision

What is LLM?

- In simpler terms, an LLM is a computer program that has been **fed enough examples to be able to recognize and interpret**

human language or other types of complex data.

Quality of the samples impacts how well LLMs will learn natural language, so an LLM's programmers may use a more curated data setقاعدة البيانات المنسقة.



LLM's And It's Limitations

- **Not Updated to the latest information:** Generative AI uses large language models to generate texts and these models have information only to date they are trained. If data is requested beyond that date, accuracy/output may be compromised.
- **Hallucinations:** Hallucinations refer to the output which is factually incorrect or nonsensical. However, the output looks coherent and grammatically correct. This information could be misleading and could have a major impact on business decision-making.
- **Domain-specific most accurate information:** LLM's output lacks accurate information many times when specificity is more important than generalized output. For instance, organizational HR policies tailored to specific employees may not be accurately addressed by LLM-based AI due to its tendency towards generic responses.
- **Source Citations:** In Generative AI responses, we don't know what source it is referring to generate a particular response. So citations become difficult and sometimes it is not ethically correct to not cite the source of information and give due credit.
- **Updates take Long training time**: information is changing very frequently and if you think to re-train those models with new information it requires huge resources and long training time which is a computationally intensive task.
- **Presenting false information** when it does not have the answer.

What is RAG?

- RAG stands for **Retrieval-Augmented Generation**
- It's an advanced technique used in Large Language Models (LLMs)
- **RAG combines** retrieval and generation processes to enhance the capabilities of LLMs
- In RAG, the model **retrieves relevant information from a knowledge base** or external sources
- This **retrieved information** is then **used in conjunction** with the model's internal knowledge to generate coherent and contextuallyمتماسكة السياق relevant responses
- RAG enables LLMs to produce **higher-quality** and **more context-aware** outputs compared to traditional generation methods
- Essentially, **RAG empowers LLMs to leverage external knowledge for improved performance** in various natural language processing tasks.

Retrieval-Augmented Generation (RAG) is an advanced artificial intelligence (AI) technique that combines information retrieval with text generation, allowing AI models to retrieve relevant information from a knowledge source and incorporate it into generated text.

Why RAG?

- "In our solar system, what planet has the most moons?“
- I know the answer from my reading experience when I was young.
- Of course, that was like 40 years ago.
- I read an article, and the article said that it was Jupiter مشتري and it have 88 moons. So that's the answer.
- Two problems in my answer:

1. I have no source to support what I'm saying.
2. I actually haven't kept up with this for awhile, and my answer is out of date.

The two behaviors are often observed as problematic in LLMs

Why RAG?

- 1- adding NASA as a source “Reference”
- 2-checking the last moth/weeks/days updates.

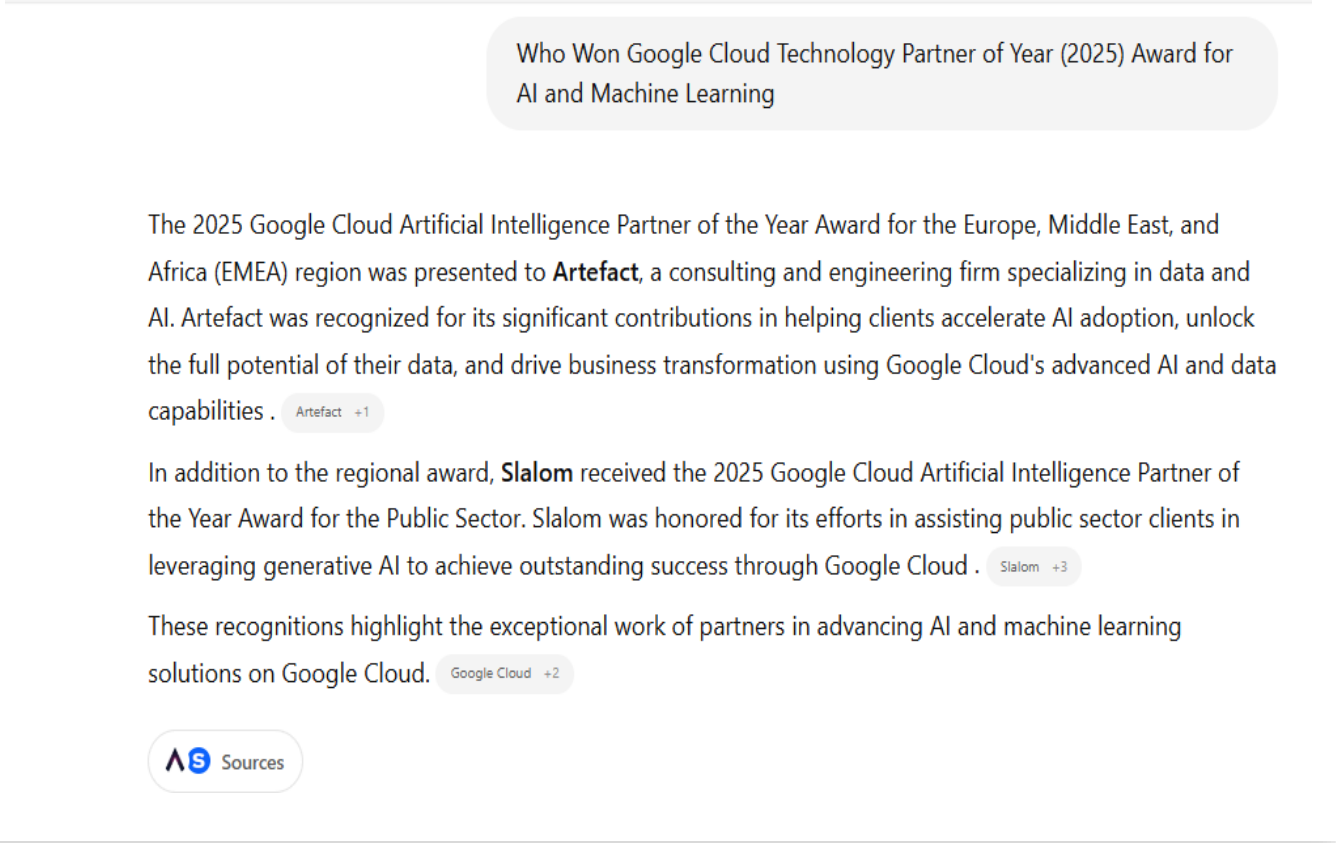
- **The answer then is :**

  Saturnزحل with 146 moons. ref “NASA”

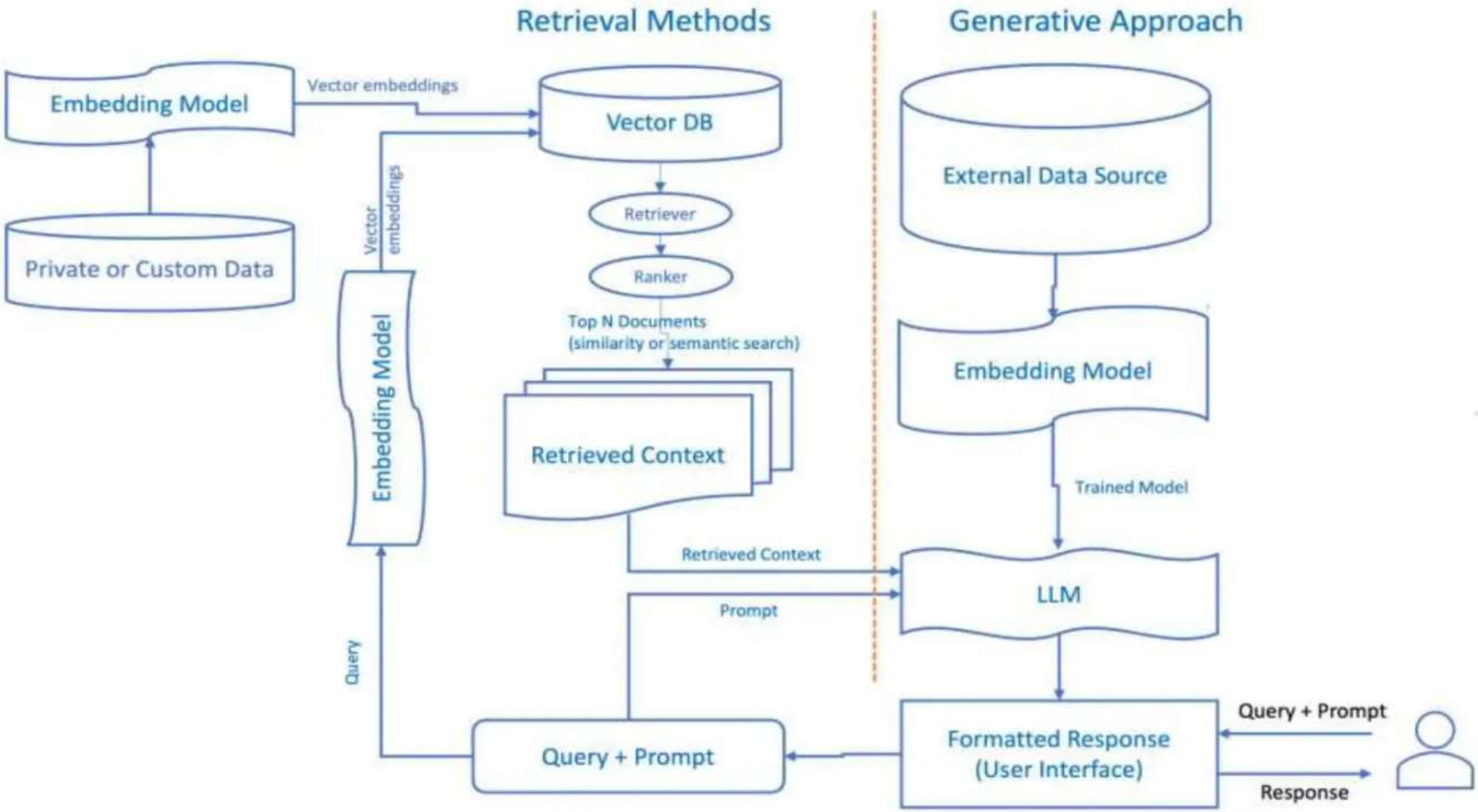
Since, **recently** many moons were discovered around Saturn.

Why is Retrieval-Augmented Generation important

- You can think of the LLM as an over-enthusiastic new employee who refuses to stay informed with current events but will always answer every question with absolute confidence.
- Unfortunately, such an attitude can negatively impact user trust and is not something you want your chatbots to emulate!
- RAG is one approach to solving some of these challenges. It redirects the LLM to retrieve relevant information from authoritative, pre-determined knowledge sources.
- Organizations have greater control over the generated text output, and users gain insights into how the LLM generates the response.



Generalized RAG Approach



How Does RAG Work?

Retrieval Augmented Generation (RAG) can be likened to a detective and storyteller duo. Imagine you are trying to solve a complex mystery. The detective's role is to gather clues, evidence, and historical records related to the case.

Once the detective has compiled this information, the storyteller designs a compelling narrative that weaves together the facts and presents a coherent story. In the context of AI, RAG operates similarly.

The Retriever Component acts as the detective, scouring databases, documents, and knowledge sources for relevant information and evidence. It compiles a comprehensive set of facts and data points.

The Generator Component assumes the role of the storyteller. Taking the collected information and transforming it into a coherent and engaging narrative, presenting a clear and detailed account of the mystery, much like a detective novel author.

This analogy illustrates how RAG combines the investigative power of retrieval with the creative skills of text generation to produce informative and engaging content, just as our detective and storyteller work together to unravel and present a compelling mystery.

RAG Components

- RAG is an AI framework that allows a generative AI model to access external information not included in its training data or model parameters to enhance its responses to prompts.
- RAG seeks to combine the strengths of both retrieval-based and generative methods.
- It typically involves using a retriever component to fetch relevant passages or documents from a large corpus of knowledge.
- The retrieved information is then used to augment the generative model's understanding and improve the quality of generated responses.
- RAG Components
  - Retriever
  - Ranker
  - Generator
  - External Data

RAG Components

- Understanding each component in detail

1. External data(mostly specific knowledge base)

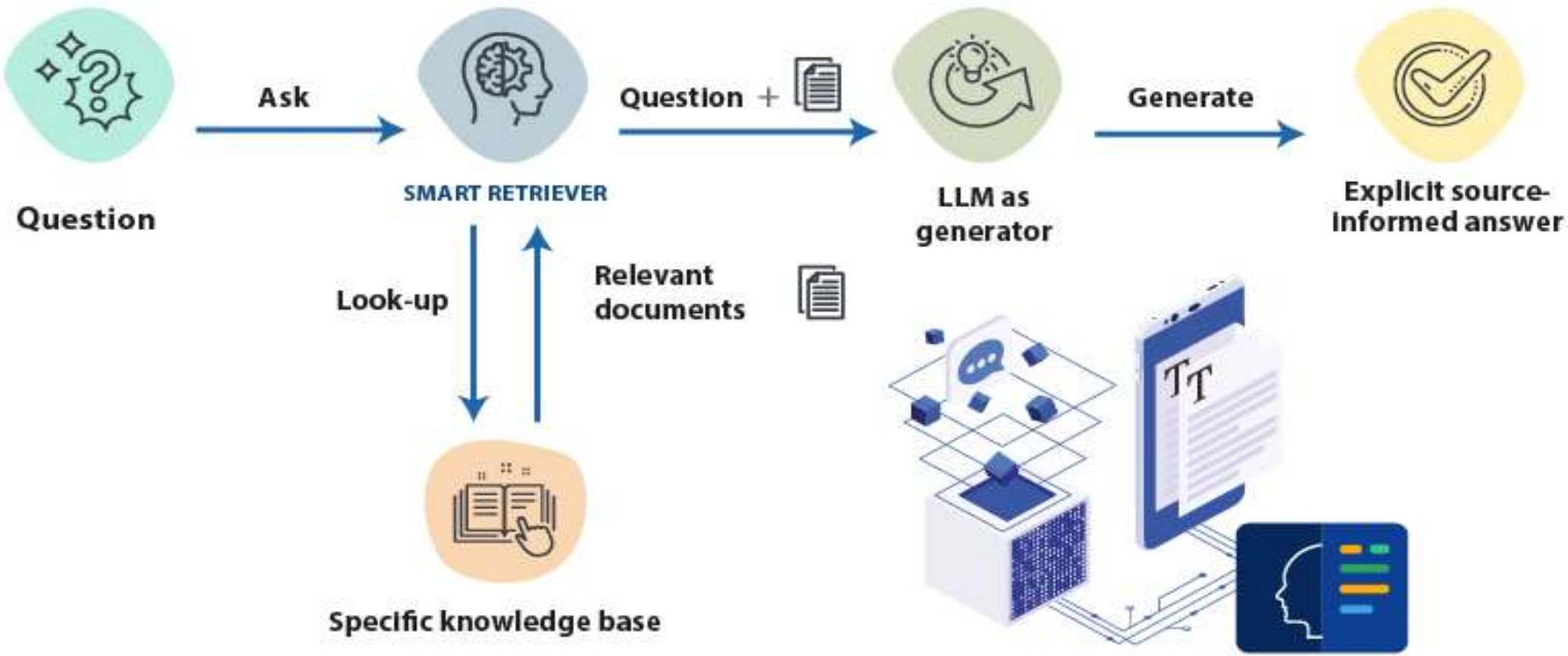
- The new data outside of the LLM's original training data set is called external data or “specific knowledge base”.
- It can come from multiple data sources, such as a APIs, databases, or document repositories. The data may exist in

RETRIEVAL-AUGMENTED GENERATION OR RAG

There are 3 phases:

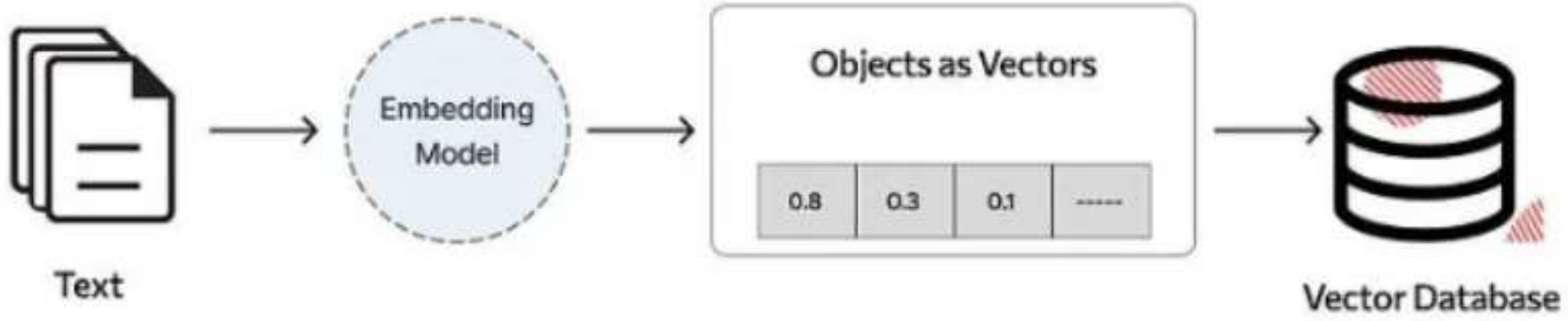
1 RETRIEVAL 2 AUGMENTED 3 GENERATION

The process is as follows:



Vector embeddings

- ML models cannot interpret information intelligibly in their raw format and require numerical data as input. They use neural
- network embeddings to convert real-word information into numerical representations called vectors.
- Vectors are numerical values that represent information in a multi-dimensional space
- Embedding vectors encode non-numerical data into a series of values that ML models can understand and relate.
- Example:
- The Conference (Horror, 2023, Movie) → The Conference (1.2, 2023, 20.0)
- Tales from the Crypt (Horror,1989, TV Show, Season 7)
  Tales from the Crypt (1.2, 1989, 36.7)
- The first number in the vector corresponds to a specific genre. An ML model would find that The Conference and Tales from the Crypt share the same genre. Likewise, the model will find more relationships



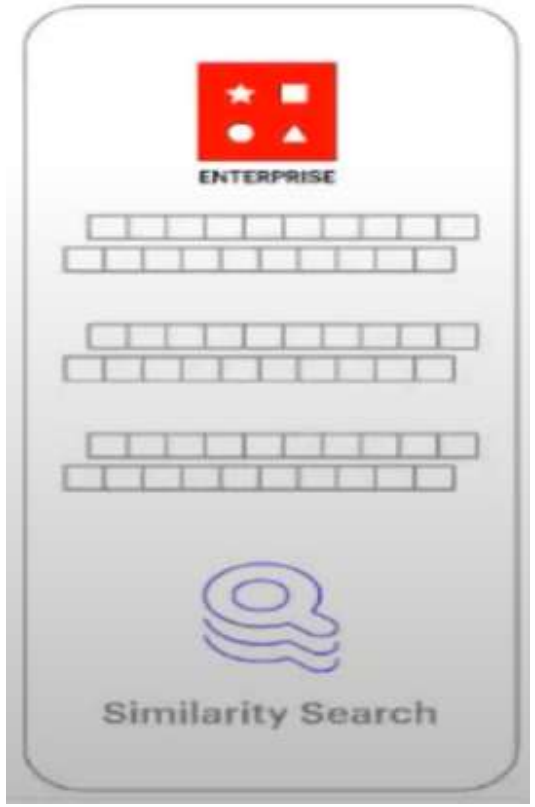
RAG components

Vector DB

- Vector DB is a database that stores embeddings of words, phrases, or documents along with their identifiers.
- It allows for fast and scalable retrieval of similar items based on their vector representations.
- Vector DBs enable efficient retrieval of relevant information during the retrieval phase of RAG, improving the contextual relevance and quality of generated
- **Data Chunking:** Before the retrieval model can search through the data, ifs typically divided into manageable "chunks" or segments.

RAG components : RAG Retriever

- The next step is to perform a relevancy search. The user query is converted to a vector representation and matched with the vector databases
- The retriever component is responsible for efficiently identifying and extracting relevant information from a vast amount of data.
- For example, consider a smart chatbot for human resource questions for an organization. If an employee searches, "How much annual leave do I have?" the system will retrieve annual leave policy documents alongside the individual employee's past leave record.
- These specific documents will be returned because they are highly-relevant to what the employee has



RAG components: RAG Ranker

- The RAG ranker component refines the retrieved information by assessing its relevance and importance. It assigns scores or ranks to the retrieved data points, helping prioritize the most relevant ones.
- The retriever component is responsible for efficiently identifying and extracting relevant information from a vast amount of data.
- For example, consider a smart chatbot that can answer human resource questions for an organization. If an employee searches, "How much annual leave do I have?" the system will retrieve annual leave policy documents alongside the individual
- employee's past leave record.

RAG components: RAG Ranker

Augment the LLM prompt

- Next, the RAG model augments the user input (or prompts) by adding the relevant retrieved data in context. This step uses
- prompt engineering techniques to communicate effectively with the LLM.
- The augmented prompt allows the large language models to generate an accurate answer to user queries.

RAG components: RAG Generator

- The RAG generator component is basically the LLM Model such a (GPT)
- The RAG generator component is responsible for taking the retrieved and ranked information, along with the user’s original query, and generating the final response or output.
- The generator ensures that the response aligns with the user's query and incorporates the factual knowledge retrieved from external sources.

RAG components: RAG Generator

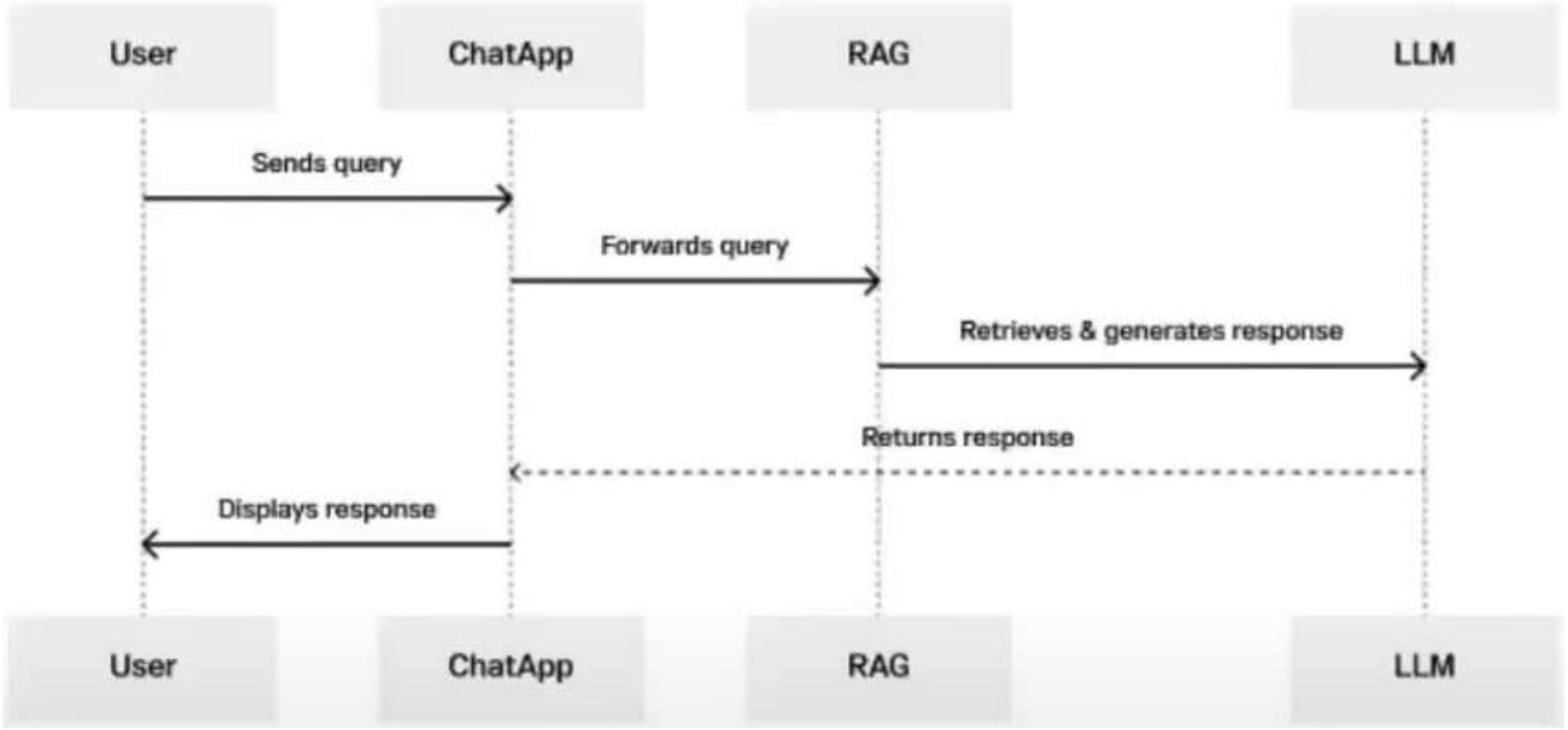
Update external data

To maintain current information for retrieval, asynchronously update the documents and update embedding representation of the documents.

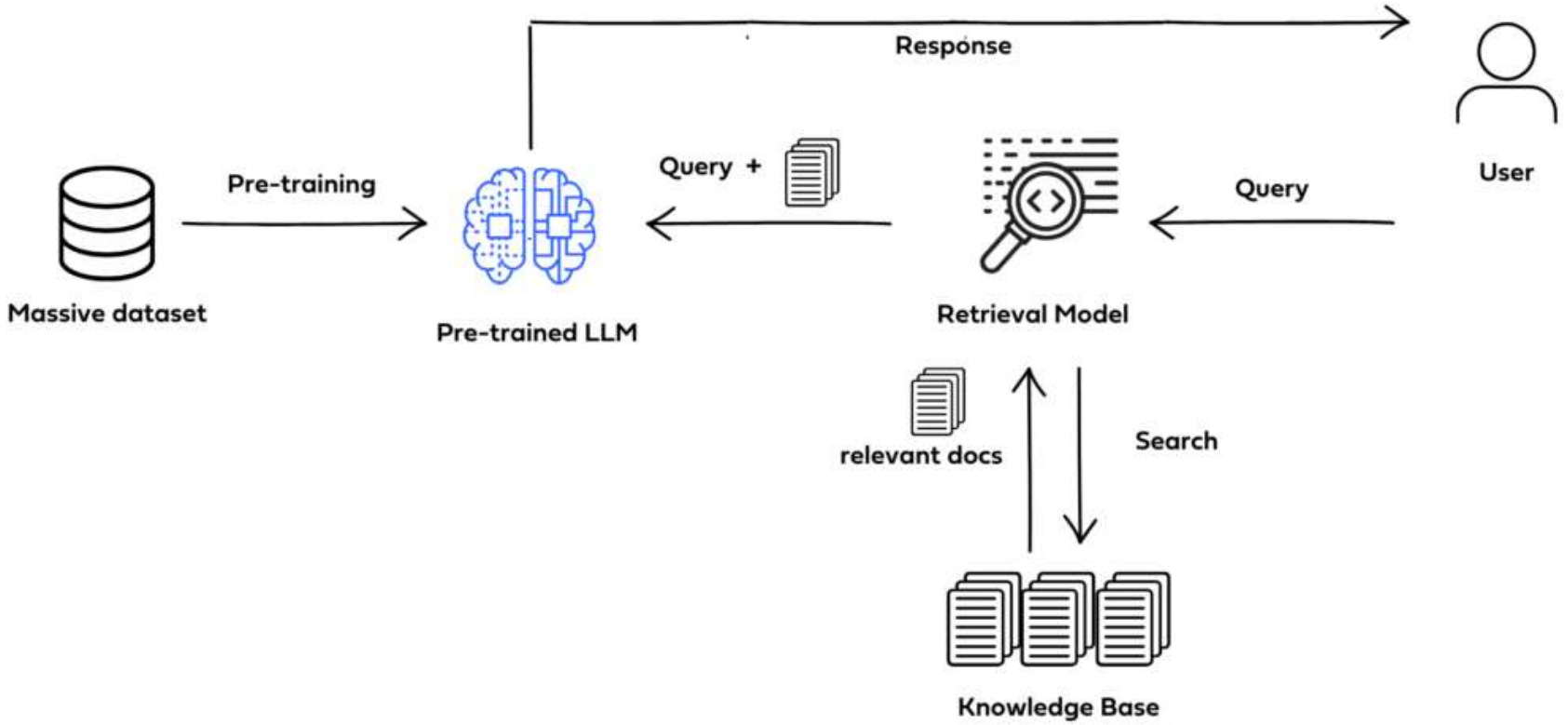
- **Automated Real-time Processes**: Updates to documents and embeddings occur in real-time as soon as new information becomes available. This ensures that the system always reflects the most recent data.
- **Periodic Batch Processing**: Updates are performed at regular intervals (e.g., daily, weekly) in batches. This approach may be more efficient for systems with large volumes of data or where real-time updates are not necessary.

RAG Based Chat Application

- Step 1 - User sends query: The process begins when the user sends a query or message to the chat application.



Retrieval Augmented Generation



Understanding RAG Architecture

- **Step 2** - Chat App forwards query: Upon receiving the user's query, the chat application (Chat App) forwards this query to the Retrieval Augmented Generation (RAG) model for processing.
- **Step 3** - RAG retrieves + generates response: The RAG model, which integrates retrieval and generation capabilities, processes the users query. It first retrieves relevant information from a large corpus of data, using the LLM to generate a coherent and contextually relevant response based on the retrieved information and the user's query.
- **Step 4** - LLM returns response: Once the response is generated, the LLM sends it back to the chat application (Chat App).
- **Step 5** - Chat App displays responses: Finally, the chat application displays the generated response to the user,

RAG Vs Fine Tuning

- **Objective**
  - Fine-tuning aims to adapt a pre-trained LLM to a specific task or domain by adjusting its parameters based on task-specific data.
  - RAG focuses on improving the quality and relevance of generated text by incorporating retrieved information from external sources during the generation process.
- **Training Data**
  - Fine-tuning requires task-specific labeled data [examples to update the model's parameters and optimize, leading to more time & cost
  - RAG relies on a combination of pre-trained LLM and external knowledge bases

RAG Vs Fine Tuning

- Adaptability
  - Fine-tuning makes the LLM more specialized and tailored to a specific task or domain.
  - RAG maintains the generalizability of the pre-trained LLM by leveraging external knowledge allowing it to adapt to a wide range of tasks.
- Model Architecture
  - Fine-tuning typically involves modifying the parameters of the pre-trained LLM while keeping its architecture unchanged.
  - RAG combines the retrieval and generation components, with the standard LLM architecture to incorporate the retrieval mechanism.

RAG Benefits

Enhanced Relevance:

- Incorporates external knowledge for more contextually relevant responses.

Improved Quality:

- Enhances the quality and accuracy of generated output.

Versatility:

- Adaptable to various tasks and domains without task-specific fine-tuning.

Efficient Retrieval:

- Leverages existing knowledge bases, reducing the need for large labeled datasets.

Dynamic Updates:

- Allows for real-time or periodic updates to maintain current information.

Trust and Transparency

- Accurate and reliable responses, underpinned by current and authoritative data, significantly enhance user trust in AI-driven applications.

Customization and Control:

- Organizations can tailor the external sources RAG draws from, allowing control over the type and scope of information integrated into the model's responses

Cost Effective

RAG Applications

Conversational AI:

- RAG enables chatbots to provide more accurate and contextually relevant responses to user queries..

Advanced Question Answering:

- RAG enhances question answering systems by retrieving relevant passages or documents containing answers to user queries.

Content Generation:

- In content generation tasks such as summarization, article writing, and content recommendation, RAG can augment the generation process with retrieved information, incorporating relevant facts, statistics, and

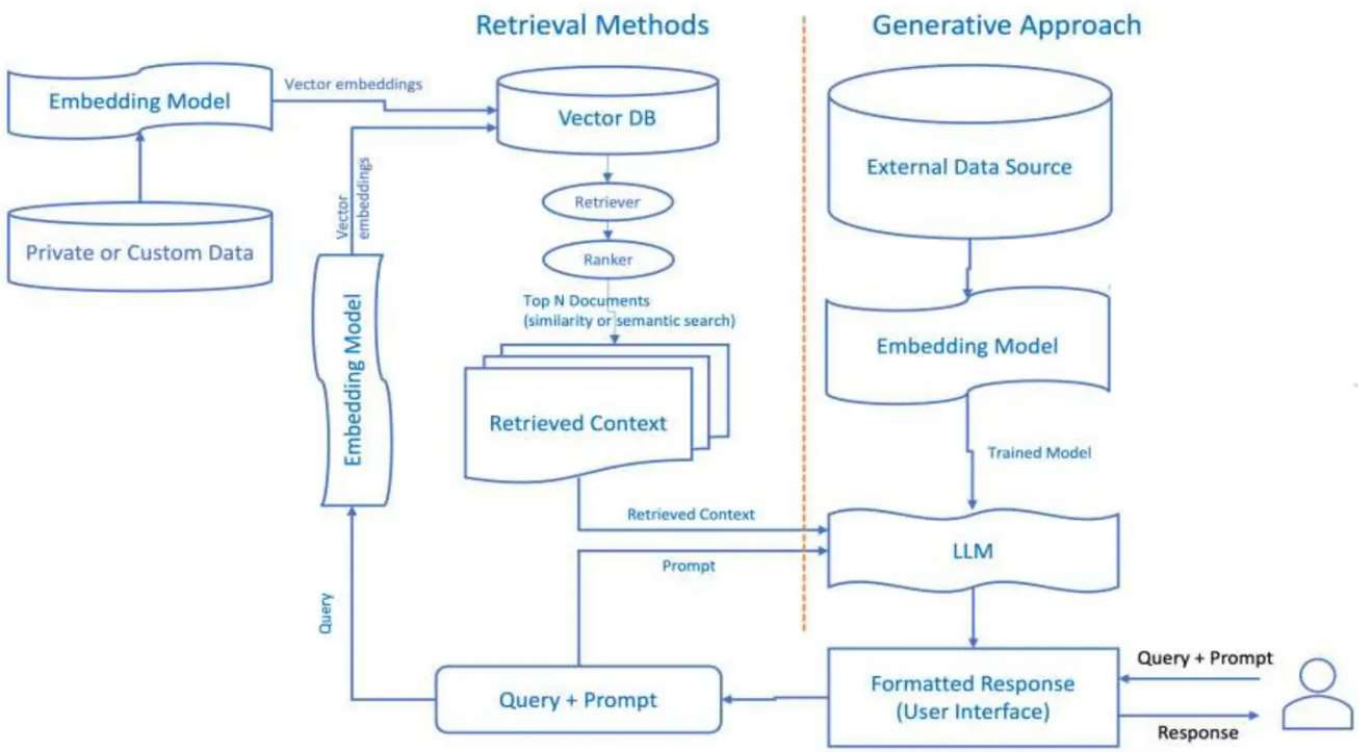
Healthcare:

- RAG can assist healthcare professionals in accessing relevant/latest medical literature, guidelines.

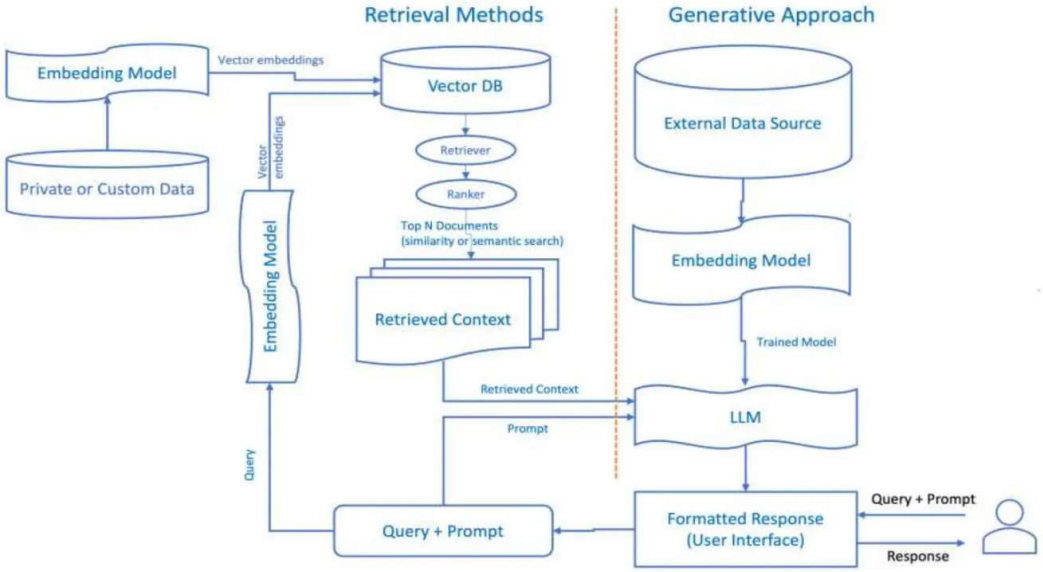
Implementing **RAG**

Step 1: Understand RAG Architecture that combines:

1. **Retriever** – Fetches relevant documents/chunks from a knowledge base.
2. **Generator** – Uses an LLM (e.g., GPT-4, Llama 2) to generate answers based on retrieved content.



Implementing **RAG**



Step 2: Set Up Components

1. Choose an LLM (Generator)

- OpenAI GPT-4, Anthropic Claude, Meta Llama 2, or open-source models like Mistral.
- Use APIs (e.g., OpenAI) or self-host (e.g., Hugging Face Transformers).

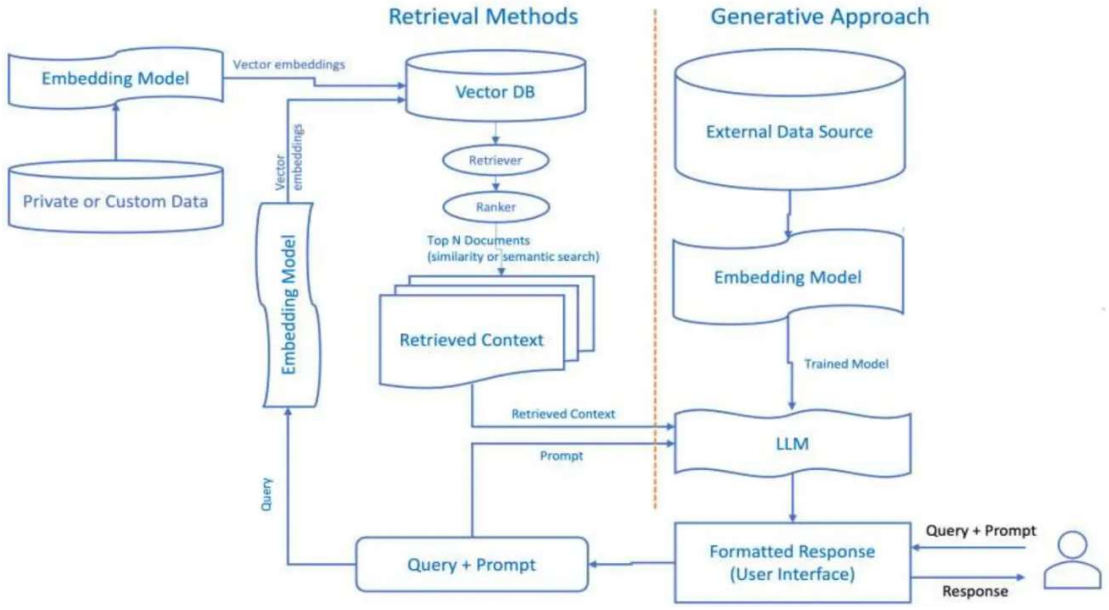
2. Select a Vector Database (Retriever)

- Stores embeddings for semantic search.
- Options: **Pinecone, Weaviate, FAISS, Chroma, Milvus**.

Implementing **RAG**

3. Embedding Model

- Converts text into numerical vectors for retrieval.
- Models: **OpenAI's text-embedding-ada-002, BERT, Sentence Transformers. (e.g., all-MiniLM-L6-v2)**



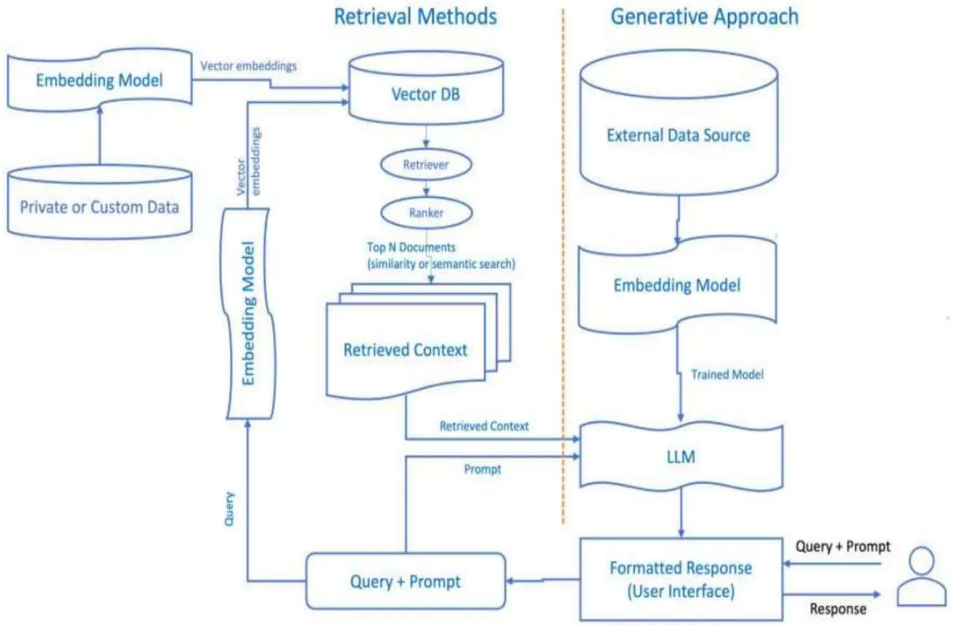
Implementing **RAG**

Step 3: Prepare the Knowledge Base

1. **Collect Data** – Gather documents (PDFs, web pages, databases).
2. **Chunk Data** – Split into smaller pieces (e.g., 256-512 tokens) for efficient retrieval.

   Recursive Character Text Splitter or Haystack.

3. **Generate Embeddings** – Convert chunks into vectors using the embedding model.
4. **Store in Vector DB** – Index embeddings for fast retrieval.

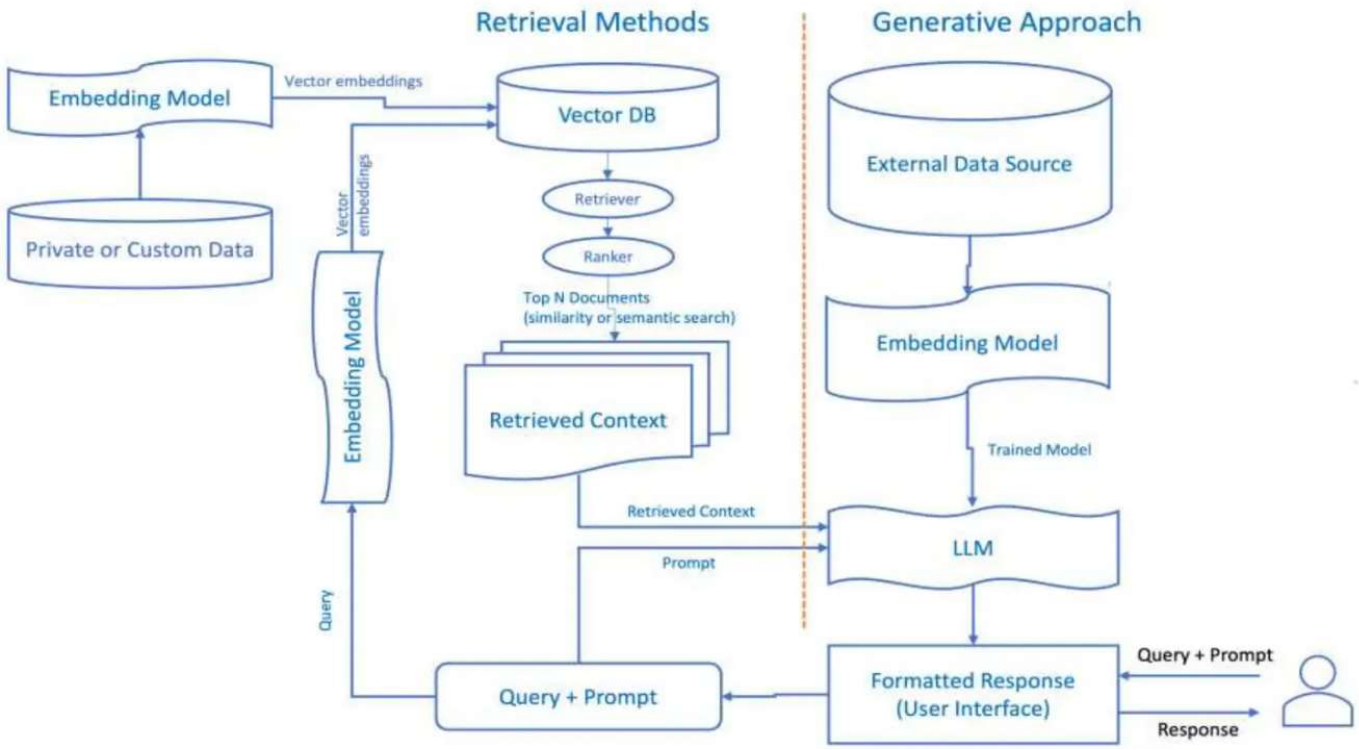


Implementing **RAG**

Step 4: Implement Retrieval

1. **User Query Processing**:
    1. Convert the query into an embedding.
2. **Semantic Search**:
    1. Retrieve top-k relevant chunks from the vector DB (e.g., cosine similarity).

- `Tools: *LangChain Retriever, LlamaIndex.*`



Implementing **RAG**

Step 5: Augment Generation

1.Pass Retrieved Context + Query to LLM:

1. Prompt template:

```
"Use the following context to answer the question:
{retrieved_text}
Question: {user_query}
Answer:“
```

Generate Response:

- The LLM synthesizes an answer from retrieved content.

Implementing **RAG**

Step 6: Build the Pipeline

Use frameworks like:

- **LangChain** (simplifies RAG pipelines):

```python
from langchain.vectorstores import FAISS
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

# Load documents → split → embed → store
loader = WebBaseLoader("https://example.com")
docs = loader.load()
embeddings = OpenAIEmbeddings()
db = FAISS.from_documents(docs, embeddings)

# RAG chain
retriever = db.as_retriever()
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    chain_type="stuff",
    retriever=retriever
)
print(qa_chain.run("What is RAG?"))
```

Implementing RAG

Step 6: Build the Pipeline
LlamaIndex (optimized for retrieval):

python

```python
from llama_index import VectorStoreIndex, SimpleWebPageReader
documents = SimpleWebPageReader(html_to_text=True).load_data(["https://example.com"])
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()
print(query_engine.query("What is RAG?"))
```

Implementing RAG

Step 7: Optimize & Evaluate

1. **Improve Retrieval**:
    1. Experiment with chunk sizes, embeddings, and hybrid search (keyword + semantic).
2. **Enhance Prompts**:
    1. Few-shot prompting or chain-of-thought (CoT) for better answers.
3. **Evaluate Metrics**:
    1. **Retrieval accuracy**: Hit rate, MRR (Mean Reciprocal Rank).
    2. **Generation quality**: BLEU, ROUGE, or human feedback.

BLEU (Bilingual Evaluation Understudy) Score **not the favorite**

ROUGE (Recall-Oriented Understudy for Gisting (conversation) Evaluation) Score **my favorite**

Implementing RAG

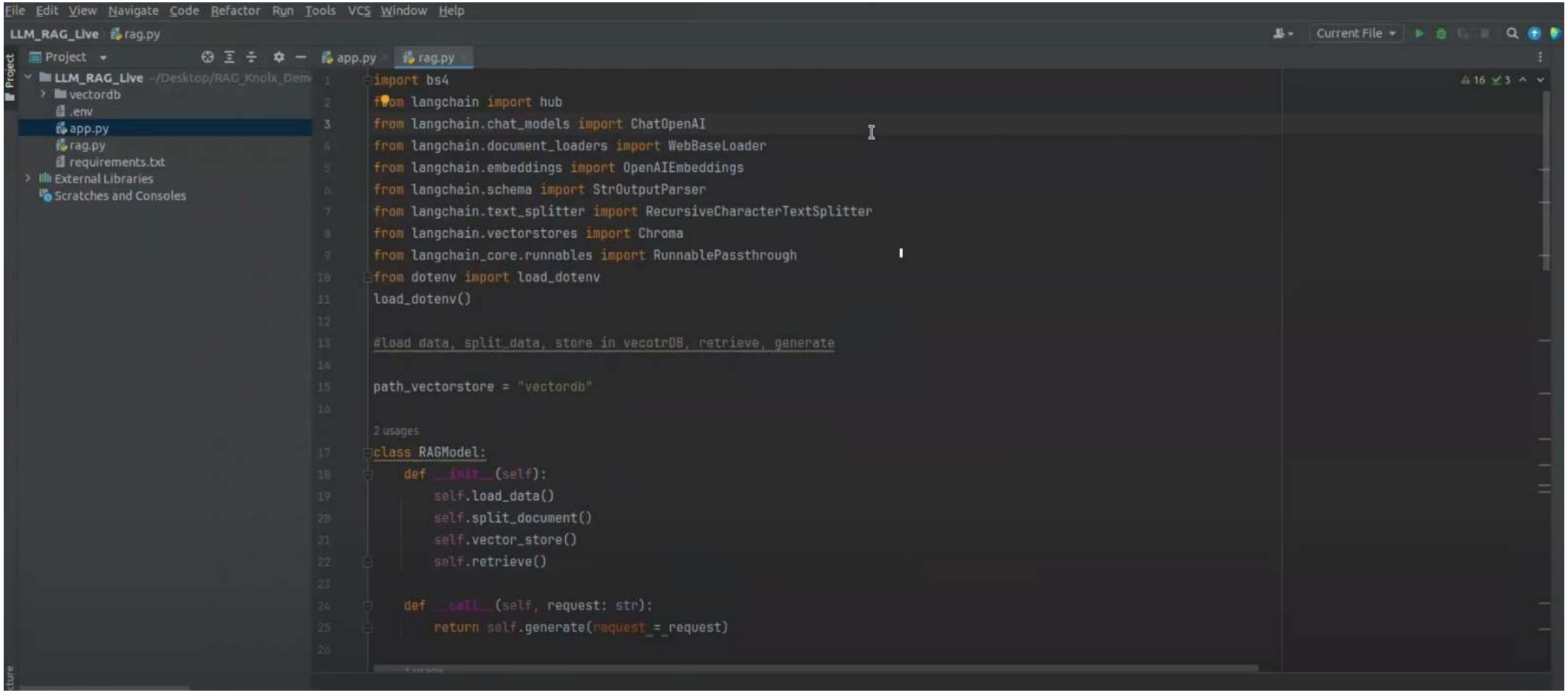
Step 8: Deploy

- **API**: Wrap in FastAPI/Flask.
- **Chat UI**: Streamlit, Gradio, or custom frontend.
- **Scalability**: Use cloud services (AWS, GCP) for vector DB and LLMs.

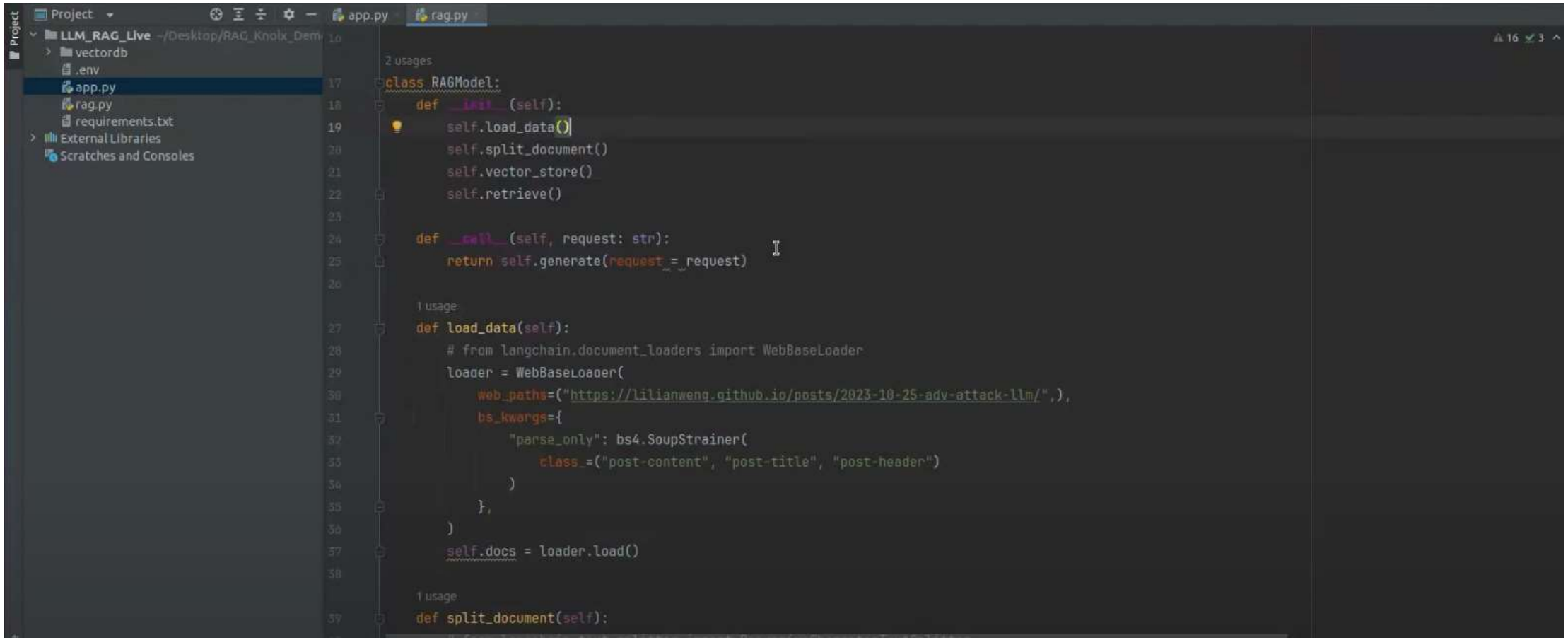
Tools & Frameworks

Component	Tools
LLMs	OpenAI, Anthropic, Llama 2, Hugging Face
Vector DB	Pinecone, Weaviate, FAISS, Chroma
Embeddings	OpenAI Ada, Sentence-BERT, Cohere
Frameworks	LangChain, LlamaIndex, Haystack

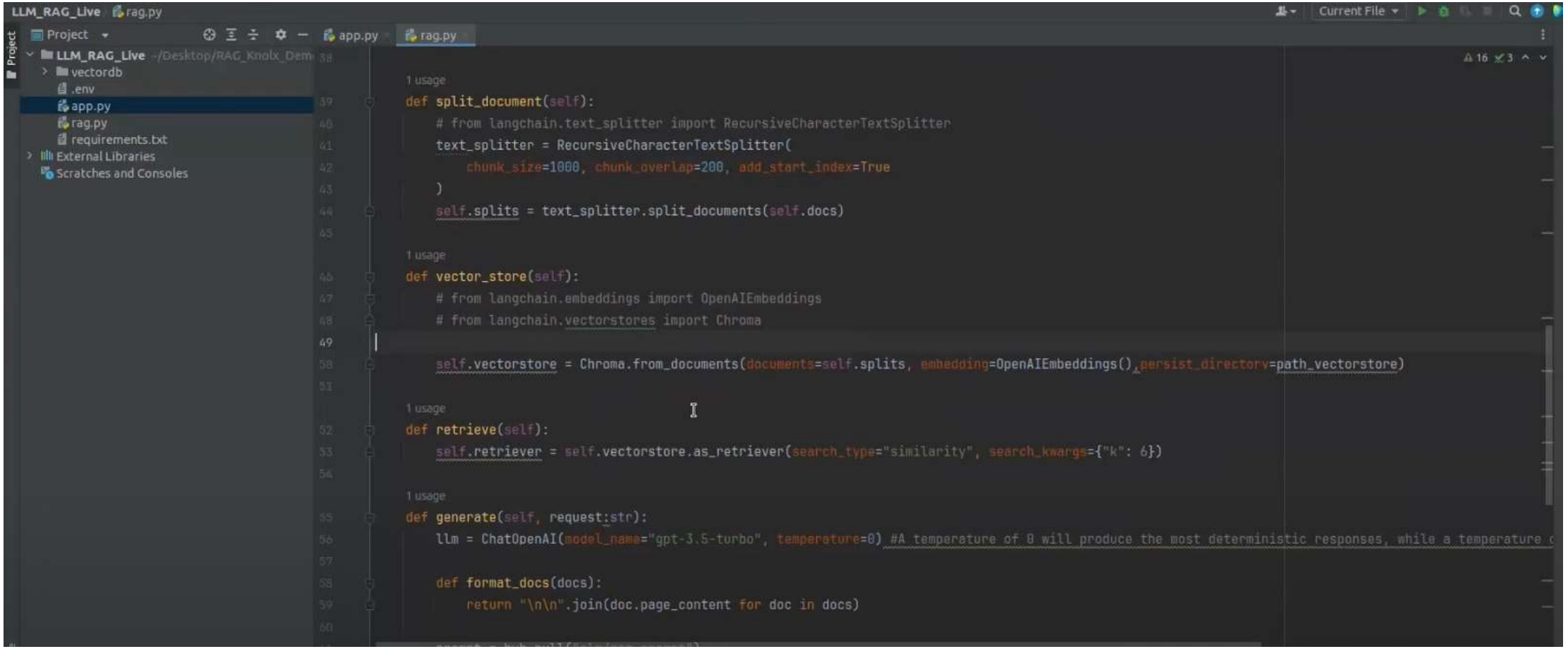
Snap shots


File Edit View Navigate Code Refactor Run Tools VCS Window Help
LLM_RAG_Live rag.py
Current File
Project
app.py
rag.py
LLM_RAG_Live ~/Desktop/RAG_Knolx_Dem
vectordb
.env
app.py
rag.py
requirements.txt
External Libraries
Scratches and Consoles
import bs4
from langchain import hub
from langchain.chat_models import ChatOpenAI
from langchain.document_loaders import WebBaseLoader
from langchain.embeddings import OpenAIEmbeddings
from langchain.schema import StrOutputParser
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import Chroma
from langchain_core.runnables import RunnablePassthrough
from dotenv import load_dotenv
load_dotenv()
#load data, split data, store in vecotrDB, retrieve, generate
path_vectorstore = "vectordb"
2 usages
class RAGModel:
def __init__(self):
self.load_data()
self.split_document()
self.vector_store()
self.retrieve()
def __call__(self, request: str):
return self.generate(request=request)

Snap shots


Project
LLM_RAG_Live ~/Desktop/RAG_Knolx_Dem
vectordb
.env
app.py
rag.py
requirements.txt
External Libraries
Scratches and Consoles
app.py
rag.py
2 usages
class RAGModel:
def __init__(self):
self.load_data()
self.split_document()
self.vector_store()
self.retrieve()
def __call__(self, request: str):
return self.generate(request_=_request)
1 usage
def load_data(self):
# from langchain.document_loaders import WebBaseLoader
loader = WebBaseLoader(
web_paths=("https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/",),
bs_kwargs={
"parse_only": bs4.SoupStrainer(
class_=("post-content", "post-title", "post-header")
)
},
)
self.docs = loader.load()
1 usage
def split_document(self):

Snap shots


LLM_RAG_Live
rag.py
Current File
Project
app.py
rag.py
LLM_RAG_Live ~/Desktop/RAG_Knolx_Dem
vectordb
.env
app.py
rag.py
requirements.txt
External Libraries
Scratches and Consoles
1 usage
def split_document(self):
# from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(
chunk_size=1000, chunk_overlap=200, add_start_index=True
)
self.splits = text_splitter.split_documents(self.docs)
1 usage
def vector_store(self):
# from langchain.embeddings import OpenAIEmbeddings
# from langchain.vectorstores import Chroma
self.vectorstore = Chroma.from_documents(documents=self.splits, embedding=OpenAIEmbeddings(),persist_directory=path_vectorstore)
1 usage
def retrieve(self):
self.retriever = self.vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 6})
1 usage
def generate(self, request:str):
llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0) #A temperature of 0 will produce the most deterministic responses, while a temperature
def format_docs(docs):
return "\n\n".join(doc.page_content for doc in docs)

CAG (**C**ontextualized **A**ugmented **G**eneration)

Cache Augmented Generation